index, the `_doc` endpoint, and the document ID. For example, we can invoke the following query to delete the document with ID 1 from the movies index.

Listing 5.35 Deleting a `movie` document from the index

```
DELETE movies/_doc/1
```

The response received from the server indicates that the document was deleted successfully:

```
{
  ...
  "_id" : "1",
  "_version" : 2,
  "result" : "deleted"
}
```

The response returns a `result` attribute set to `deleted` to let the client know the object was deleted successfully. If the document isn't deleted (for instance, it doesn't exist in our store), we get a response with the `result` value set to `not_found` (status code 404). Interestingly, Elasticsearch increments the `_version` flag if the delete operation succeeds.

5.5.2 Deleting by query (_delete_by_query)

Deleting a single document is easy, as we just saw. But if we want to delete multiple documents based on a criterion, we can use `_delete_by_query` (similar to `_update_by_query` in section 5.4.7). If we want to delete all movies directed by James Cameron, for example, we can write this query.

Listing 5.36 Deleting all movies based on a criterion

```
POST movies/_delete_by_query
{
  "query":{
    "match":{
      "director":"James Cameron"
    }
  }
}
```

Here we create the query in the request body using as our criterion all movies directed by James Cameron. Documents that are matched with the criterion are then marked and deleted.

The body of this `POST` uses a special syntax called Query DSL (domain-specific language), which lets us pass in a variety of attributes like `term`, `range`, and `match` (as in this listing), similar to basic `search` queries. We learn more about search queries in later chapters, but for now, note that `_delete_by_query` is a powerful endpoint with sophisticated delete criteria. Let's see a few examples in the following sections.